EclipseStack Technical Report

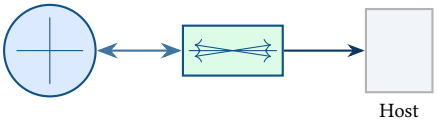
Architecture, Tooling, and Implementation Notes

Simon Knight

Adelaide, Australia

March 2026 • Version 0.1.0

Last updated: March 11, 2026



Abstract. This technical report documents EclipseStack's architecture, build and release workflow, and implementation decisions. It is intended as a living engineering reference that captures system boundaries, key components, and rationale for design choices.

Contents

List of Design Decisions & Rules

1 Overview

This report follows the `sk-techreport` format from `/dev/papers`. Use it to record architectural intent and the reasoning behind changes.

Design Decision 1: Scope

This document focuses on: system architecture, module boundaries, build/test tooling, and design decisions that impact maintainability.

2 System Architecture

Describe the high-level architecture here, including the main subsystems and their interfaces.

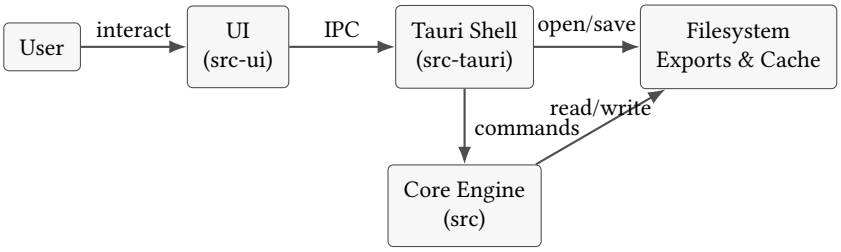


Figure 1. High-level component boundaries and data flow.

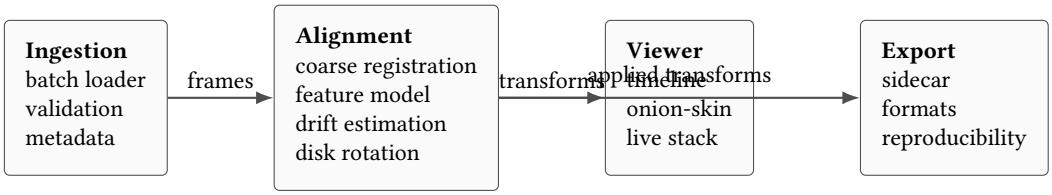


Figure 2. Core processing pipeline from ingestion to export.

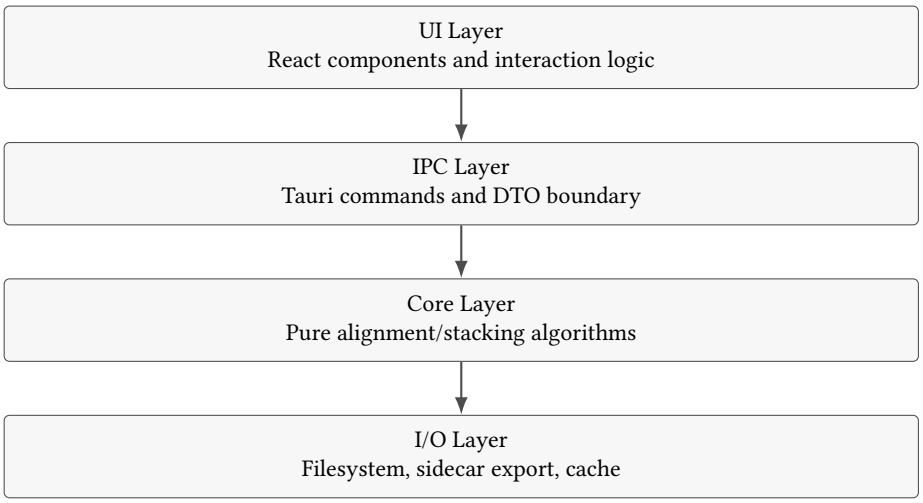


Figure 3. Layering model: keep algorithms independent from UI concerns.

Design Decision 2: Repository Layout

Record why directories such as `src`, `src-tauri`, and `src-ui` exist, and how responsibilities are split.

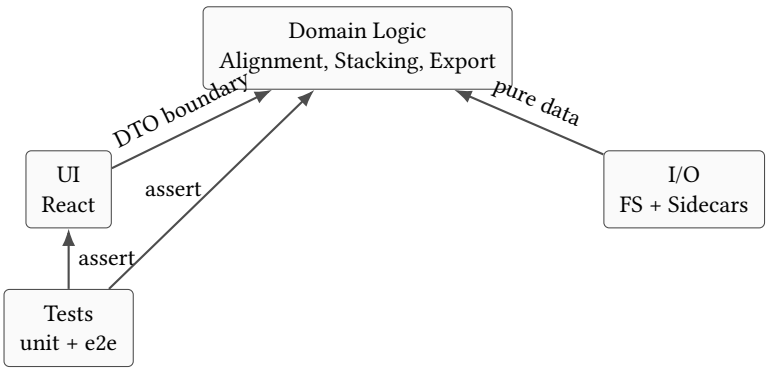


Figure 4. Dependency direction: UI and I/O depend on domain logic, not the reverse.

3 Build, Test, and Release

Document build commands, CI expectations, and release steps.

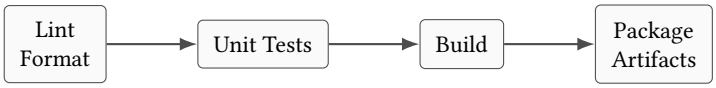


Figure 5. Typical developer and CI pipeline for changes.

Figure 6. Change management path from local edits to release artifacts.

: Reproducible Builds

Builds should be reproducible on developer machines and CI by pinning toolchains and documenting required dependencies.

3.1 Math and Algorithm Notes

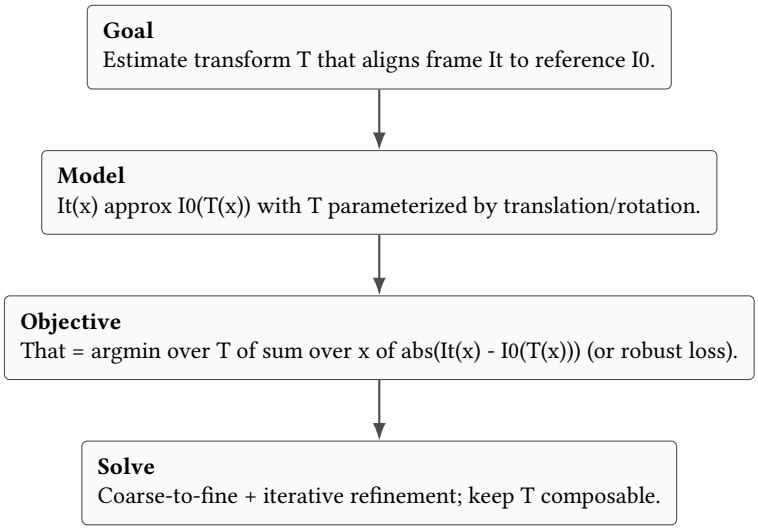


Figure 7. Generic alignment formulation used throughout the pipeline.

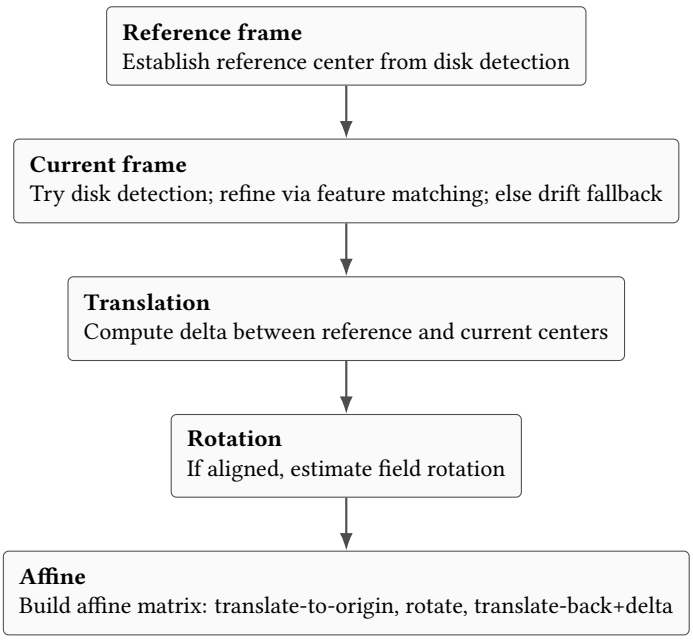


Figure 8. Transform construction flow as implemented by the alignment pipeline.

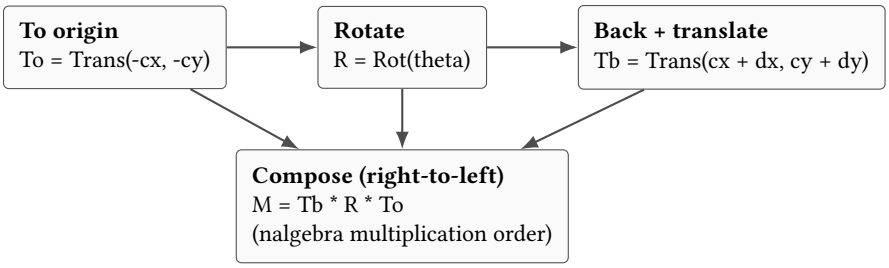


Figure 9. Affine composition used by the transform builder.

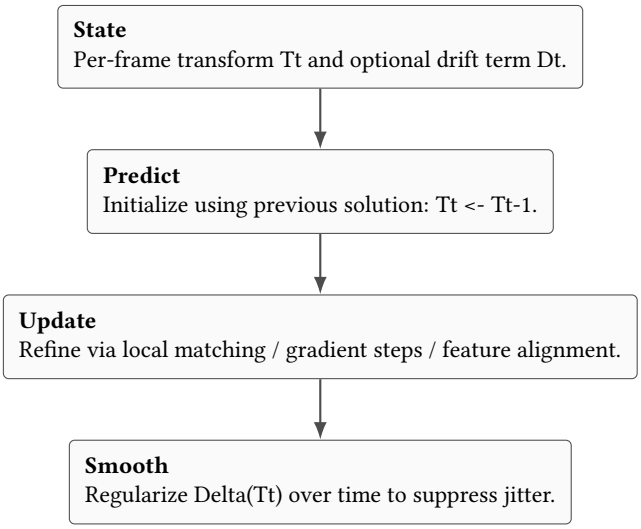


Figure 10. Temporal tracking pattern: propagate, refine, then regularize.

Figure 11. Feature-based alignment: detect, match, fit, verify.

3.2 Code Logic and Data Shapes

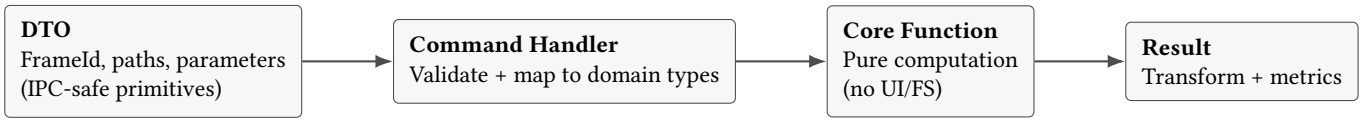


Figure 12. Code boundary: IPC DTOs map into pure core functions and back.

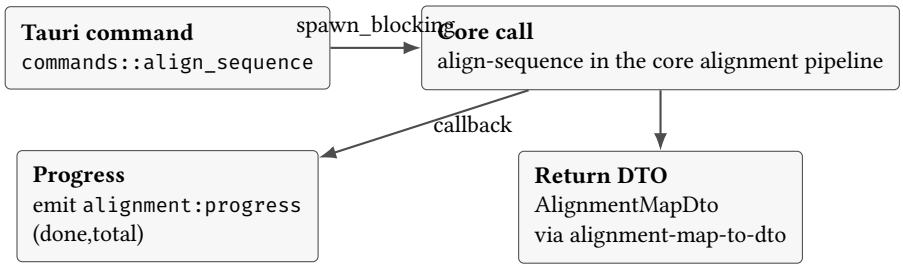


Figure 13. UI-to-core alignment flow: command, progress events, and DTO serialization.

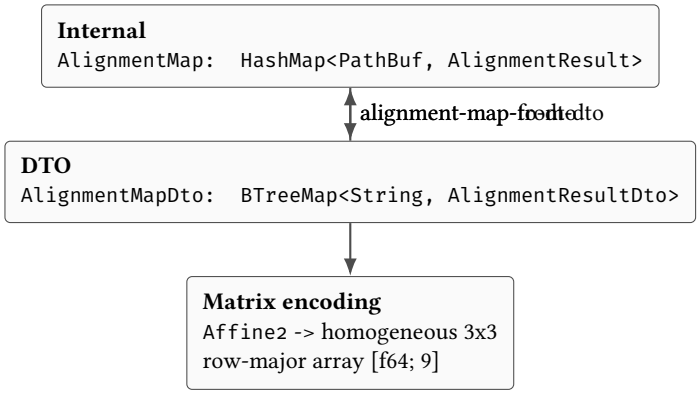


Figure 14. IPC/persistence data model: deterministic key ordering and explicit matrix layout.

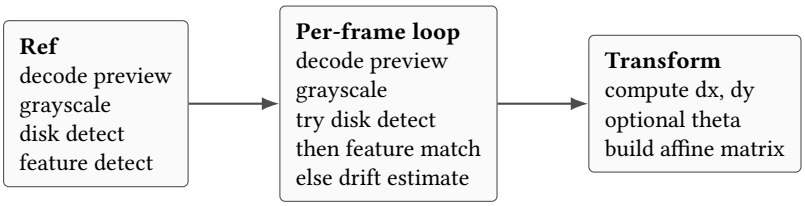


Figure 15. Control flow inside the core alignment pipeline.

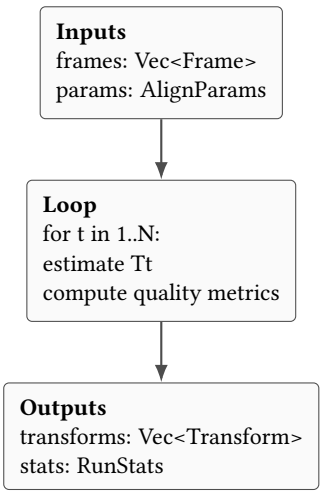


Figure 16. Core alignment function shape: batch inputs, per-frame loop, batch outputs.

3.3 Module-Level Logic (Alignment)

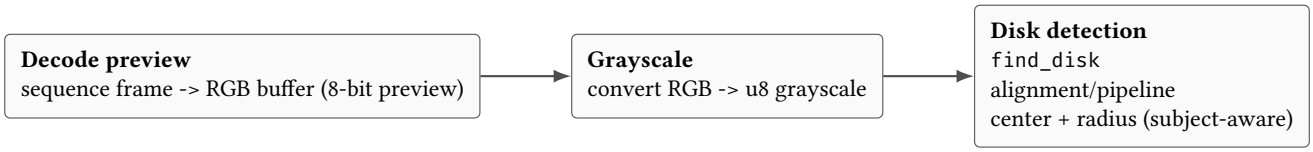


Figure 17. Early-stage preprocessing in the alignment pipeline: decode, grayscale, detect disk.

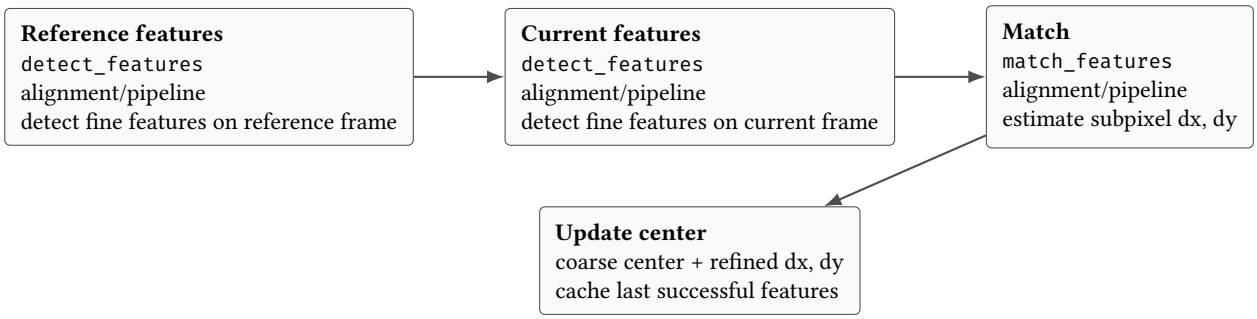


Figure 18. Fine alignment using feature detection and matching (translation refinement).

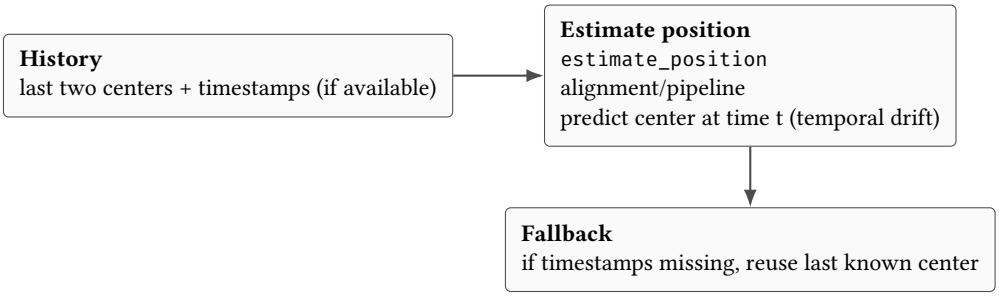


Figure 19. Temporal drift fallback when disk detection and feature matching fail.

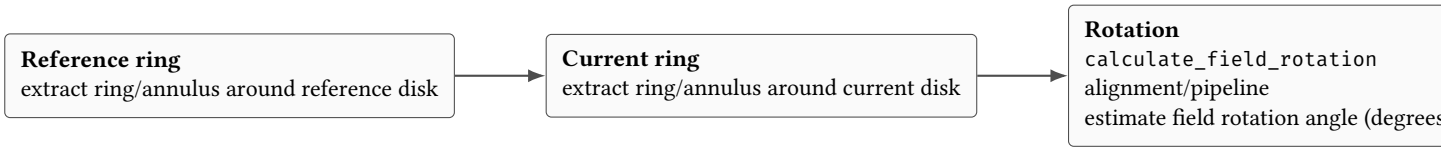


Figure 20. Field rotation estimation (only when coarse alignment succeeds).

3.4 Worked Example (Transform Construction)

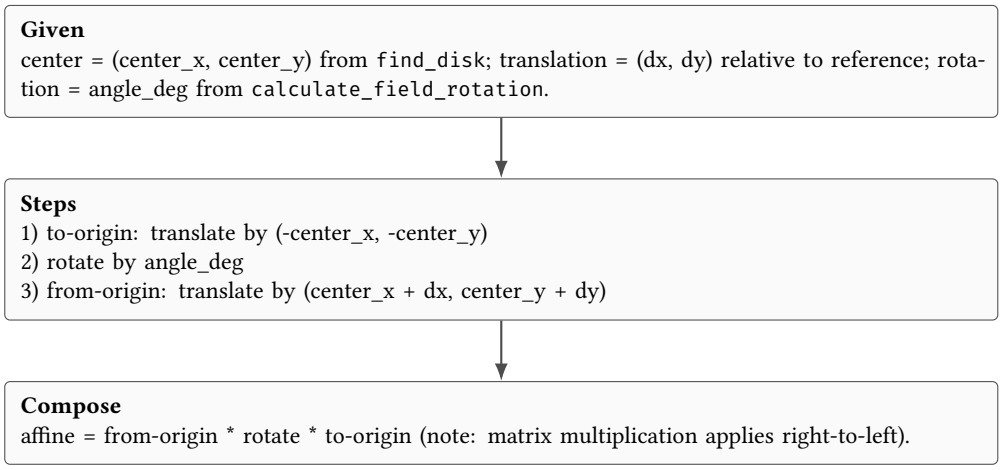


Figure 21. Worked example: how a per-frame affine transform is constructed.

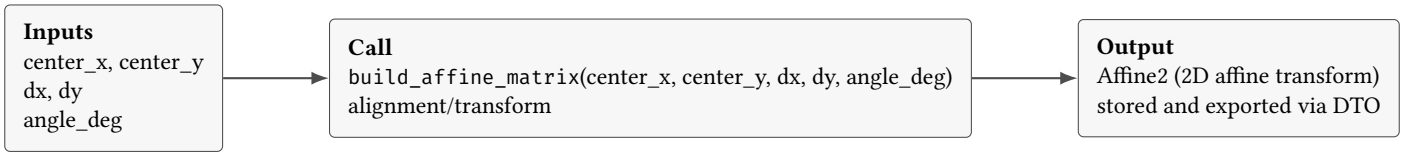


Figure 22. Implementation view of the transform builder: inputs and output shape.

4 Design Decisions

Add design decisions as they are made. Keep each one small and focused.

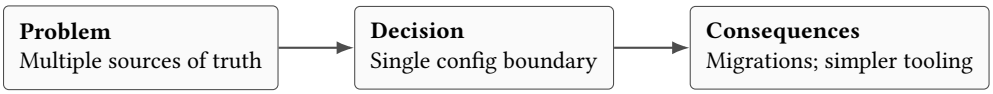


Figure 23. Decision record structure: problem, decision, consequences.

Design Decision 3: Example Decision

Decision: Choose a single source of truth for configuration.
Rationale: Avoid divergence between multiple config formats.
Consequences: Some migrations may be required.

List of Figures

List of Tables